

Why Your MLP Failed & Recovery Options

Root Cause Analysis: MLP Failure

The Problem Wasn't MLP Architecture

Your MLP architecture (16 features → hidden layers → output) is actually **well-suited** for ED complexity:

-  Can capture non-linear feature interactions
-  Can adapt to changing patterns
-  Can discover unknown relationships
-  Can handle temporal dynamics

The Problem Was Target Definition

```
python
```

```
# What your MLP was trained to predict:
```

```
target = "will_be_admitted" # Binary outcome
```

```
# Problems with this target:
```

1. Admission depends on bed availability (**not** medical need)
2. Admission depends on social factors (nursing home placement)
3. Admission depends on time of day (bed manager availability)
4. Admission depends on insurance/policy factors
5. Many "**should be admitted**" patients go home due to capacity

```
# Result: Model learns noise, not signal
```

```
threshold = 0.9988 # Model says "almost never admit"
```

MLP Recovery Options

Option A: Fix the Target Variable

Instead of predicting admission, predict **medical complexity** or **clinical risk**:

python

Better targets for MLP:

```
target_options = [  
    "medical_complexity_score", # 0-10 scale of medical needs  
    "clinical_deterioration_risk", # Probability of getting sicker  
    "resource_intensity_needed", # How much care/monitoring needed  
    "time_to_medical_stability", # Hours until medically stable  
    "readmission_risk_7day" # Likelihood of bounce-back  
]
```

These are medically meaningful, not operationally confounded

Option B: Multi-Target MLP

Train MLP to predict **multiple clinically relevant outcomes**:

python

```
class MultiTargetED_MLP(nn.Module):  
    def forward(self, features):  
        shared_layers = self.backbone(features) # 16 → hidden  
  
        outputs = {  
            'clinical_risk': self.risk_head(shared_layers), # 0-1 prob  
            'complexity_score': self.complexity_head(shared_layers), # 1-10 scale  
            'stability_eta': self.stability_head(shared_layers), # hours  
            'resource_needs': self.resource_head(shared_layers) # categorical  
        }  
        return outputs
```

Option C: Hybrid Architecture (Best of Both)

python

Stage 1: MLP for clinical assessment

```
clinical_assessment = mlp.predict(features)
```

→ medical_complexity: 7/10, deterioration_risk: 0.23

Stage 2: Gate engine for operational workflow

```
operational_assessment = gate_engine.compute_readiness(  
    features, clinical_assessment, context  
)
```

→ priority: 8/10, next_action: "ICU consult", eta: 120min

Combined intelligence:

- MLP captures non-linear clinical patterns

- Gate engine handles operational logic

Architectural Comparison: What Each Excels At

MLP Strengths (Why You Originally Chose It)

python

Excellent for:

✅ **Complex feature interactions**

- IF (age > 65 AND troponin > 50 AND hr > 100) → high risk
- Non-obvious combinations that rules miss

✅ **Temporal pattern learning**

- Weekend vs weekday differences
- Seasonal variations (flu season)
- Time-of-day effects

✅ **Continuous adaptation**

- New protocols → automatic adaptation
- Staff changes → learns new patterns
- Equipment updates → adjusts thresholds

✅ **Pattern discovery**

- Finds hidden correlations in data
- Discovers early warning signs
- Identifies subtle clinical combinations

Example: MLP might discover that

```
# (t_min=45 AND MAP=85 AND Lab_pending=True AND Leitsymptom="dyspnoe")
```

predicts deterioration, even when individual va

